

План курса

Литература

- Б.В. Керниган, Д.М. Ричи. «Язык программирования C»
- Python books: <https://wiki.python.org/moin/PythonBooks>
- Бьерн Страуструп «Язык программирования C++»



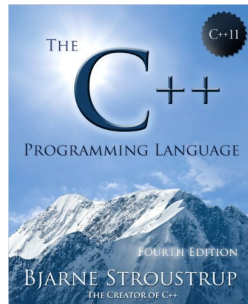
python™

» PythonBooks

» IntroductoryBooks » PythonBooks

There are a variety of books about Python. Here's a guide to them:

- » [IntroductoryBooks](#) (gentle overviews of the language)
- » [AdvancedBooks](#) (for when you don't want gentle)
- » [ReferenceBooks](#) (much information in a small space)
- » Specific applications:
 - » [GameProgrammingBooks](#)
 - » [NetworkProgrammingBooks](#)
 - » [GuiBooks](#)
 - » [JythonBooks](#)
 - » [ScientificProgrammingBooks](#)
 - » [SystemAdministrationBooks](#)
 - » [WebProgrammingBooks](#)
 - » [WindowsBooks](#)
 - » [XmlBooks](#)
 - » [ZopeBooks](#)
- » Books in languages other than English:



Лекции и практические задания

- 👉 Лекции на GitHub
- 👉 Лекции «внутри ОИЯИ»

Для практических занятий требуется:

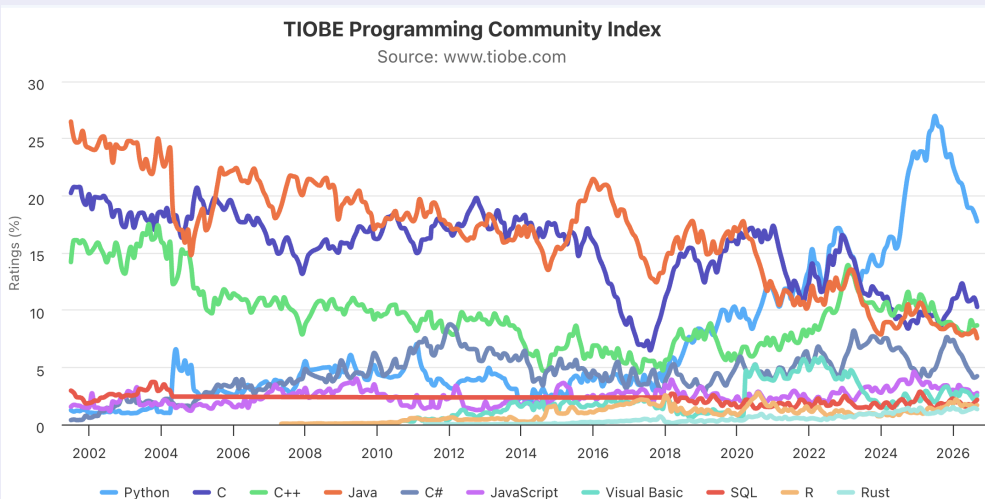
- Компилятор `gcc` (GNU compiler collection) или `clang` («клэнг»)
- Python-3

Рекомендации для персонального компьютера

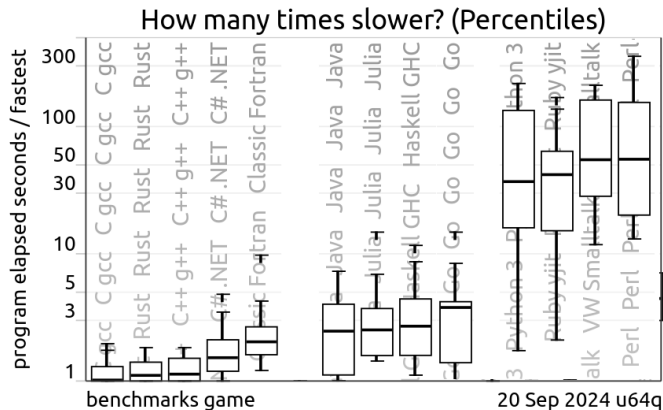
- ✓ Либо Unix подобная ОС: Linux, macOS ...
- ✓ Либо Windows и одно из возможных решений:
 - 1 Инсталлировать какую-то Linux ОС под Windows Subsystem for Linux (WSL)
 - 2 Установить VirtualBox for Windows host и затем скачать Ubuntu собранное для VirtualBox, см. <https://www.osboxes.org/faqs/>

Место Python/C/C++ среди других языков

«Индекс популярности» языков



«Тест производительности» языков, Benchmarks Game



- несколько программ для «типичных проблем»

👉 тесты это не предсказания!

Краткая история стандартов языка C

- **C89** – первый официальный стандарт C: ANSI 1989
(C90 – first ISO version, C95 – bug-fix, minor version)
- **C99** – ISO версия 1999 года: **улучшена поддержка численных расчетов**
- **C11, C17** – предыдущие ревизии, C17 is a bug-fix of C11
- **C23** – текущая стабильная версия, принята 31 октября 2024
улучшена совместимость с C++
- **C29** – следующая ожидаемая версия

Поддержка компиляторами «последних версий»

- **GCC:** gcc-15 по умолчанию C23 (**-std=c2y для C29**)
- **clang:** clang-19 частично с **-std=c23** (по умолчанию C17)
- **Microsoft Visual:** частично C99 и начиная с VS 2019: C11(std:c11) и C17(std:c17)

Пример на C: сумма ряда $\cos 0^\circ + \cos 1^\circ + \dots + \cos 90^\circ$

```
#include <stdio.h>
#include <math.h>
int main() {
    const double gr2rad = M_PI/180; /* degrees to radians */
    double sum=0;
    for(int i = 0; i <= 90; i++) { // C99
        sum += cos(i*gr2rad);
    }
    char math_fmt[] = u8" 90°\n Σ cos(i) = %f\ni=0°\n"; // C11
    [[maybe_unused]] auto np = printf(math_fmt,sum); // C23
}
```

- Файл: `sum_cos.c` (`.c` – стандартное расширение для C)
- Компиляция: `clang -std=c23 sum_cos.c -lm`
`-std=c23` (или `-std=gnu23`) – использовать стандарт языка C23
`-lm` – «подключить» математическую библиотеку

👉 Выполнение: `./a.out`

$$\sum_{i=0^\circ}^{90^\circ} \cos(i) = 57.794325$$

Краткий обзор языка

Основные типы данных

<code>char</code>	— один байт, в котором находится один символ
<code>int</code>	— целое, размер зависит от используемой машины
<code>float</code>	— число с плавающей точкой одинарной точности
<code>double</code>	— число с плавающей точкой двойной точности
<code>bool</code>	— логический тип в C23
<code>nullptr_t</code>	— тип константы <code>nullptr</code> C23

Инициализация: используйте явную инициализацию где возможно

```
int i,j,k;           // uninitialized variables
char let_A = 'A';    // explicit initialization
int CamelCaps = 1;    // горбатый регистр
long int Very_long_names_like_this_do_not_make_sense = 0L;
double zero = {};     // empty initialization C23 (нулем)
```

- тип переменной определяется по типу правой части

```
auto a = 1.2;      // a - double
auto b = 1;        // b - int
auto c = true;     // c - bool
auto x = fun(b);   // x - тип который возвращает fun(int)
```

☞ обеспечивает «некоторую» совместимость с C++

Сравнение auto в C23 и C++

```
// no multiple declaration support in C23
// auto a1 = sin(1.), a2 = sin(2.); // C23 ERROR; C++ OK
auto a1 = sin(1.); auto a2 = sin(2.); // both OK
printf("a1= %f a2= %f\n",a1,a2); // a1= 0.841471 a2= 0.909297
// can be used for pointers, but C23 does not allowed 'auto *'
const char* test="Hello world";
auto pw = strchr( test, 'w' ); // both OK: char*
// auto* pw2 = strchr( test, 'w' ); // C23 ERROR; C++ OK
printf("pw= %p, *pw= %c\n",pw,*pw); // pw= 0x10455c64a, *pw= w
```

Оператор `sizeof(obj)` — возвращает размер объекта `obj` в «байтах»

1 байт $\equiv$ `sizeof(char)`

X86-64

- `void*` = 8
- `short int` = 2
- `int` = 4
- `long int` = 8
- `long long int` = 8
- `float` = 4
- `double` = 8
- `long double` = 16
- `bool` = 1 // C23
- `nullptr_t` = 8 // C23

ARM64

- `void*` = 8
- `short int` = 2
- `int` = 4
- `long int` = 8
- `long long int` = 8
- `float` = 4
- `double` = 8
- `bool` = 1 // C23
- `nullptr_t` = 8 // C23

`sizeof(long long)` $\geq$ `sizeof(long)` $\geq$ `sizeof(int)` $\geq$ `sizeof(short)`
`sizeof(long double)` $\geq$ `sizeof(double)` $\geq$ `sizeof(float)`

Операторы `typeof` C23

- `typeof(expression)` — возвращает тип выражения
- `typeof_unqual(expression)` — тип выражения без «квалификаторов»
- Оператор `typeof` похож на оператор `decltype` в C++11

```
auto x = 0.1; // double
const auto cx = 0.2; // const double
typeof(x) y = 0; // type of y is the same as x
typeof_unqual(cx) v; // the same as cx but without 'const'
v = cx + 1; // OK: v is not constant
printf("%.1f %.1f %.1f %.1f\n",x,cx,y,v); // 0.1 0.2 0.0 1.2

typeof(int*) p1 = nullptr, p2 = nullptr; // both pointers to int
printf("%p %p\n",p1,p2); // 0x0 0x0

// pointer to function (double) returning double
typeof(double(double)) *pf = sin;
printf("sin(0.1)=%f\n",pf(0.1)); // sin(0.1)=0.099833
```

Область видимости, score

Область видимости переменной — часть программы в которой эту переменную можно использовать

Глобальная область видимости

Переменные объявленные вне функций доступны в любом месте программы

- все функции по умолчанию глобальные
- глобальные переменные

Локальная область видимости

Внутри блока между { и }:

- переменная существует от места объявления до конца блока
- параметры функции являются локальными переменными в теле функции

Пример: глобальные и локальные переменные

```
int f_counter = 0; /* global variable */
double a = 1.;    /* other global variable */

double fun(double x) {
    f_counter++;
    double a = 10; /* local in the fun() variable */
    return ret = fabs(x-a); /* |x-a| */
}

int main() {
    fun(a); /* using global a */
    int a = 5; /* local in the main() variable a */
    fun(a); /* using local a */
    printf("function has been called %d times\n", f_counter);
}
```

Операторы

Присваивание

```
a = b;
```

```
c = d = e = f = 0;
```

```
2 = k; /* Ошибка! */
```

👉 **Lvalue (left-value)** — то, что может стоять в левой части оператора присваивания

Арифметические унарные операторы: $-$, $+$

```
b = -a;      /* меняет знак на противоположный */
```

```
c = +a;      /* не выполняет никаких действий */
```

Бинарные операторы: +, −, *, / и % (операция деления по модулю)

```
int a = 1, b = 2, c = 3;  
a = b + c;           /* a=5 */  
b = a / c;           /* b=1 */  
c = a % 3;           /* c=2 */
```

👉 Операция % только для целых типов

«Гибридные» операторы: $e1 \odot= e2$; то же, что $e1 = (e1) \odot (e2)$;

```
counter_i += 10;      /* counter_i = counter_i + 10; */  
i %= j;               /* i = i % j; i,j - int! */  
a *= x + 1;           /* a = a * (x+1); */  
a + b *= 2;           /* Ошибка! В левой части должно стоять Lvalue */  
val[i1+i2] *= 2;      /* Правильно: val[i1+i2]=val[i1+i2]*2 */
```

Операторы ++ и --, инкремент/декремент

Увеличивают/уменьшают величину на единицу и возвращают значение:

- **Префиксные**, оператор перед переменной: ++i; --i;

j = ++i;	=>	i = i + 1;
		j = i;

- **Постфиксные**, оператор после переменной: i++; i--;

j = i++;	=>	j = i;
		i = i + 1;

Запрещено или надо избегать:

```
j = ++i++;          // ERROR: i++ or ++i is not lvalue
j = ++i*i++;        // WARNING: operation on 'i' may be undefined
j = f(++i,++i);     // a more general case of the previous
```

```
auto i=1; printf("%i %i %i\n",i,++i,i++); // gcc: 3 3 2; clang: 1 2 2
```

Операции отношения и логические операции

- Операции: отношения `>= > <= <` , равенства и неравенства `== !=` , отрицание (унарный оператор): `!`
- Логические связки: `&&` («И») `||` («ИЛИ»)
- ☞ До C23 вместо «логического типа» используются целочисленные переменные: `0` – ложь (`FALSE`), `1` (не ноль!) – истина (`TRUE`)
- ☞ В C23 `bool` – встроенный логический тип и `true`, `false` – две предопределенные константы

```
if( n%2 == 0 && n%3 == 0 ) {  
    printf(" %d divisible by 6 without a remainder\n",n);  
}  
if( b=fun(a) ) { /* it's not a bug */  
    printf(" fun(%d) returns a non-zero value %d\n",a,b);  
}  
bool a = true > false; // C23  
printf("C23: true = %d, false= %d, a= %d\n",true,false,a); // 1 0 1
```

Обратите внимание

- $0 < j < 10$ то же самое, что $(0 < j) < 10$ и это всегда TRUE
☞ Правильная запись: $0 < j \ \&\& \ j < 10$
- Результаты логических выражений можно использовать как обычные целые числа:

```
int minus_one_in_power(int i) { //  $(-1)^i$   
    return -1 + 2*(i%2 == 0);  
}
```

- Часть логической связки с $\&\&$, $\|$ может не выполняться

☞ Так делать не надо:

```
if( (a >= 0 &\& a <= 10) &\& ++b > 0 ) {...}
```

Условный троичный оператор:

`(condition) ? (expression-true) : (expression-false) ;`

- 1 Вычисляется `condition`
- 2 Если условие истинно, то вычисляется `expression-true`, в противном случае `expression-false`
- 3 Вычисленное значение возвращается, значит оператор может стоять в правой части присваивания

```
bool a = true > false;
printf("a=%s\n", (a ? "true" : "false")); // a=true

int m = (n > 0) ? -n : n;    // == -abs(n)

int max(int a, int b) {
    return (a > b) ? a : b;
}
```

Цикл for

```
for(ExprIni; Cond; ExprIter) {  
    statements;  
}
```

«ПОЧТИ» for

```
ExprIni;          // initial stmt.  
while(Cond) {     // condition  
    statements;   // loop body  
    ExprIter;     // «iteration»  
}
```

```
for(i = 0; i < 10; i++) {...}
```

- ❶ Инициализация перед выполнением цикла: $i = 0$;
- ❷ Условие продолжения цикла, первая инструкция цикла: $i < 10$;
- ❸ Тело цикла: $\{ \dots \}$
- ❹ Изменение счётчика, последняя инструкция цикла: $i++$;

C99 допускает декларацию переменной в первом поле for

```
for ( int i = 0; i < n; i++ ) { ... }
```

👉 Область видимости этой переменной: **только внутри цикла**

- Любое из трёх выражений в **for** можно оставить пустым, но разделители (**;;**) всегда должны присутствовать
- «Пустое условие» в **for** равносильно бесконечному циклу

Примеры организации бесконечного цикла:

`for (;;) {...}` // с циклом **for**

`while(1) {...}` // с циклом **while**; в C23 и C++ **while(true)**

- ☞ Для выхода из бесконечного цикла используют операторы: **break** или **return** и очень редко **goto**

Операторы перехода управляющие поведением цикла

- 1 оператор **break** немедленно прекращает выполнение **текущего** цикла
- 2 оператор **continue** прерывает итерацию и переходит к следующей:
☞ в цикле **for** переходит на изменение итератора

Оператор goto

- Для оператора `goto` всегда необходима метка – идентификатор с последующим двоеточием: например `LOOP:`
- Управление передается на ту точку программы, где стоит метка
- Метка может находиться как до, так и после оператора `goto`

```
int b[5] = {1,2,3,2,1};  
int sum = 0;  
int i = 0;  
LOOP:;  
    sum += b[i++];  
    if( i < 5 ) { goto LOOP;} // сравните с циклом do-while  
printf("sum= %d\n",sum); // sum= 9
```

 Старайтесь не использовать `goto`

Старайтесь не использовать goto

- В структурном программировании **goto** рассматривают как крайне нежелательный оператор. *«Качество программного кода обратно пропорционально количеству операторов goto в нём»*
(E.W. Dijkstra "Go to statement considered harmful", 1968)
- «Спагетти-код» – программа содержащая много операторов **goto**

Оправданное применение: завершение многократно вложенных циклов
Donald Knuth, "Structured Programming with go to Statements", 1974

```
for( int i = 0; i < n; i++ )  
    for( int j = 0; j < m; j++ )  
        if( matrix[i][j] == value ) {  
            printf("value %d found in cell (%d,%d)\n",value,i,j);  
            goto END_LOOP;  
        }  
printf("value %d not found\n",value);  
END_LOOP: ;
```

Дополнительные слайды

Имена переменных: идентификаторы

- Все переменные должны быть декларированы до их использования
- Имена состоят из букв, цифр и «подчёркиваний»; цифра не должна быть первой
- ПРОПИСНЫЕ и строчные буквы разные!
- Число значимых символов: 31(63) внешние(внутренние) имена
- Имена должны отличаться от служебных слов

Комментарии в C

```
/*  
    1) Traditional  
    multi-line comments in C  
*/  
  
// 2) single-line comments in C99, like in C++
```

```
c = 1 + d /* - 123 */ + k; // 3) combining comments
```

Точка с запятой ; *(end of statement)*

```
x = 0;           // признак конца оператора
;               // пустой оператор
for(i=0; i<10; sum=i++); // тоже пустой оператор
x = sin(a)       // выражение можно
+               // разбить на
cos(b);         // несколько линий
```

Фигурные скобки { ... } *(compound statement)*

☞ Используются для объединения описаний и операторов в составной оператор или блок

```
if ( a < b ) {
    a++;
    b--;
}
```

Оператор последовательного вычисления (запятая ,)

`expression1 , expression2 , expression3 , ...`

- Выражения, разделённые запятой, вычисляются слева направо
- Возвращается значение правого (последнего) операнда

Пример:

```
for ( i = 1, j = 9; i <= 5; i++, j-- ) { ... }
```

не имеют отношения к операции запятая

- запятые разделяющие переменные в декларациях
`int i=1, j=0;`
- запятые разделяющие параметры или аргументы функций
`double f(int x, int y);`
`x += f(i, j);`

Оператор 'if-else' используется при необходимости сделать выбор

```
if (логическое выражение)
    оператор-1
else
    оператор-2          // необязательная
                        // часть
```

Пример: $z = \max(a, b)$

```
// 1-st variant
z = a;
if(b > z) {z = b;}
```

```
// 2-nd variant
if (a > b) {
    z = a;
} else {
    z = b;
}
```

👉 Конструкция для выбора из нескольких вариантов

1 'if-else-if'

```
if (expression-1) {  
    statement;  
}  
else if (expression-2) {  
    statement;  
}  
else if (expression-3) {  
    statement;  
}  
else {  
    statement;  
}
```

2 Переключатель switch

```
switch (expression) {  
    case Const_Int_1:  
        statement;  
    case Const_Int_2:  
        statement;  
    ...  
    default:  
        statement;  
}
```

Особенности switch

- Значение `expression` должно быть целым: `int`, `char` или `bool` в C23
- При совпадении выражения с одним из `case` переходит на него и выполняется **до конца switch** или до оператора `break`
- Если выражение не совпадает ни с одним из `case`, переходит на `default`
- `default` может отсутствовать

Цикл while

```
while (expression) {  
    statement;  
}
```

- цикл выполняется по тех пор пока **expression** "истина"
- затем выполняется следующий за **while** оператор

Цикл do - while

```
do {  
    statement;  
} while (expression);
```

- **statement** выполняется в любом случае
- вычисляется **expression** и цикл повторяется если результат "истина"

Оператор break

- 1 В цикле: немедленно прекращает выполнение **текущего** цикла
- 2 В операторе **switch**: осуществляет переход на конец оператора **switch**

Оператор continue

- Прерывает итерацию цикла и переходит к следующей итерации
☞ в цикле **for** переход на приращение **ExprIter**

☞ во вложенных циклах и **break** и **continue** управляют только циклом в котором стоят

```
for (i = 0; i < 5; i++) {  
    for (j = 0; j < 5; j++) {  
        if(j+i>2) break;  
        printf("(%d,%d)",i,j);  
    }  
}
```

> (0,0)(0,1)(0,2)(1,0)(1,1)(2,0)

```
for (i = 0; i < 5; i++) {  
    for (j = 0; j < 5; j++) {  
        if(j+i<6) continue;  
        printf("(%d,%d)",i,j);  
    }  
}
```

> (2,4)(3,3)(3,4)(4,2)(4,3)(4,4)